

Game

```
import time
from DBConn import *
from Game import *
from Snake import *

class Game:
    def __init__(self):
        curses.initscr()
        self.win = curses.newwin(WINDOW_HEIGHT, WINDOW_WIDTH, 0,
0)

        self.win.keypad(True)
        curses.noecho()
        curses.curs_set(0)
        self.win.border(0)
        self.win.nodelay(True)
        self.key = curses.KEY_RIGHT

    def getKey(self):
        prev_key = self.key
        event = self.win.getch()
        if (event == curses.KEY_LEFT and prev_key ==
curses.KEY_RIGHT) or \
            (event == curses.KEY_RIGHT and prev_key ==
curses.KEY_LEFT) or \
            (event == curses.KEY_UP and prev_key ==
curses.KEY_DOWN) or \
            (event == curses.KEY_DOWN and prev_key ==
curses.KEY_UP):
            event = prev_key

        self.key = event if event != -1 else prev_key

        if self.key not in [curses.KEY_LEFT, curses.KEY_RIGHT,
curses.KEY_UP, curses.KEY_DOWN, ESC]:
            self.key = prev_key

    def setPos(self, snake):
        y = snake.tails[0].y
        x = snake.tails[0].x
        if self.key == curses.KEY_DOWN:
```

```

        y += 1
    if self.key == curses.KEY_UP:
        y -= 1
    if self.key == curses.KEY_LEFT:
        x -= 1
    if self.key == curses.KEY_RIGHT:
        x += 1

    return y, x

def start(self):
    score = 0
    snake = Snake(self.win)
    food = Food(self.win, snake)

    while self.key != ESC:
        self.win.addstr(0, 2, ' Score ' + str(score) + ' ')
        self.win.timeout(150 - (len(snake.tails)) //
                        5 + len(snake.tails)//10 % 120)

        self.getKey()

        y, x = self.setPos(snake)

        snake.moveHead(y, x)

        if snake.collison():
            break

        if snake.eat(food.pos):
            score += 1
            food.spawn()
        else:
            snake.move()

        snake.draw()

    curses.endwin()
    return score

def menu():
    title = [

```

```

"-----" /
" | | \ | | | ----- | | / /"
,
" | ----- | | \ | | | | | | / /" ,
" | | | \ | | | | ----- | | / /" ,
" | | ----- | | | | ----- | | /" ,
" | ----- | | \ | | | | | | \\" ,
" ----- | | | | \\" ,
\\ | | | ----- | | \ \ \\" ,
" | | | | \ | | | | | | |
\\ \\" ,
" | ----- | | ----- | | ----- | | ----- \
\" ,
"=====
```

```
for i in title:
    time.sleep(0.1)
    print(i)
print(" MAIN MENU")
print(" > 1. Play")
print(" > 2. Highscore")
print(" > 3. Exit")
return int(input(" Enter number: "))
```

jadi di file game ini, memiliki beberapa library yaitu time, Dbconn, game, dan snake

jadi untuk library time ini fungsinya agar snake nya jalan berdasarkan waktu yang sudah ditentukan, dan dbconn ini fungsinya untuk menghubungkan database mysql nya

class game digunakan untuk mendefinisikan

def get key digunakan untuk menerima input dari keyboard dan input itu akan diteruskan dan mengubah arah ular nya bergerak

det setpos ini mengubah suatu inputan menjadi gerakan dari ular

def start fungsinya untuk mereset score, snake, dan foodnya. Selama user tidak menekan tombol esc maka game akan terus berlanjut sampai ular menabrak dinding. Collision membuat saat menabrak itu akan menghentikan code. Eat membuat ukuran ular bertambah ketika makan

Main

```
import os
from Game import *
from tabulate import tabulate
```

```

while True:
    os.system("cls")
    selected = menu()
    if selected == 1:
        score = Game().start()
        os.system("cls")
        print("==== Game Over ====")
        print("> Final score:", score)
        name = input("> Enter your name: ")
        Score().insert(name, score)
    elif selected == 2:
        os.system("cls")
        data = Score().getAllData()
        print("=====")
        print("|           High Score           |")
        if len(data) == 0:
            print("=====")
            print("|           No Data           |")
            print("=====")
        else:
            print(tabulate(data, headers=["Pos", "Name",
"Score"], tablefmt="psql", showindex=range(1, len(data)+1)))
            os.system("pause")
    elif selected == 3:
        print("=====")
        print(" Good Bye ~")
        os.system("pause")
        os.system("cls")
        break

```

jadi disini memiliki 3 library yaitu os, game dari file dan tabulate yang bisa di download menggunakan cmd. Selected menu digunakan untuk variabel yg diisi oleh menu. apabila user memilih 1 maka game akan dimulai, jika user memilih 2 maka akan menunjukkan high score sedangkan option ke tiga akan keluar dari aplikasi

DBConn

```
import mysql.connector

class Score:
    def __init__(self):
        self.__mydb = mysql.connector.connect(
            host="localhost",
            user="root",
            password="",
            database="snake"
        )
        self.__mycursor = self.__mydb.cursor()

    def getAllData(self):
        self.__mycursor.execute("SELECT name, value FROM
score ORDER BY value DESC LIMIT 10")
        return self.__mycursor.fetchall()

    def insert(self, name, score):
        sql = "INSERT INTO score (name, value) VALUES (%s,
%s)"

        val = (name, score)
        self.__mycursor.execute(sql, val)
        self.__mydb.commit()
```

jadi code DBConn ini digunakan untuk menghubungkan database mysql. Disini perlu menginput nama 'snak' untuk database nya di dalam mysql

fungsi GetAllData ini untuk mengambil skor kolom nama yang dimana skor ini memiliki limit yaitu 10. Fungsi Insert untuk menginput nama dan skor

SNAKE

```
import curses
from random import randint

WINDOW_WIDTH = 60
WINDOW_HEIGHT = 20
ESC = 27
```

```
class Tail:
    char = '*'

    def __init__(self, y, x):
        self.setTail(y, x)

    def setTail(self, y, x):
        self.y = y
        self.x = x
        self.pos = (y, x)

class Snake():
    head = ()

    def __init__(self, win):
        self.tails = []
        self.win = win
        self.addEntry(4, 2)
        self.addEntry(4, 3)
        self.addEntry(4, 4)

    def collision(self):
        tail = []
        y = self.tails[0].y
        x = self.tails[0].x
        for tails in self.tails:
            tail.append(tails.pos)

        if y == 0 or y == WINDOW_HEIGHT - 1 or \
            x == 0 or x == WINDOW_WIDTH - 1 or \
            self.head in tail[1:]:
            curses.flash()
            curses.beep()
            return True
        else:
            return False

    def eat(self, food):
        return self.head == food
```

```

def move(self):
    last = self.tails.pop()
    self.win.addch(last.y, last.x, ' ')

def draw(self):
    self.win.addch(self.tails[0].y, self.tails[0].x, '*')

def moveHead(self, y, x):
    self.addEntry(y, x)

def addEntry(self, y, x):
    self.head = (y, x)
    self.tails.insert(0, Tail(y, x))

class Food():
    def __init__(self, win, snake):
        self.snake = snake
        self.win = win
        self.setPos(6, 6)
        win.addch(self.y, self.x, '#')

    def setPos(self, y, x):
        self.y = y
        self.x = x
        self.pos = (y, x)

    def spawn(self):
        curses.flash()
        self.pos = ()
        while self.pos == ():
            self.setPos(randint(1, WINDOW_HEIGHT-2),
                        randint(1, WINDOW_WIDTH - 2))
        for tail in self.snake.tails:
            if self.pos == tail.pos:
                self.pos = ()
        self.win.addch(self.y, self.x, '#')

```

jadi disini memiliki 3 class yaitu tail, snake, dan food

jadi di dalam kelas tail itu hanya untuk mengset tail nya dan posisinya tail nya

dan untuk kelas snake itu menjelaskan gimana cara snake itu berjalan dan jika kena pinggir dia akan mati, dan jika kena badannya sendiri maka dia akan mati, dan di dalam kelas ini juga menjelaskan gimana gerak nya snake ini.

Dan untuk kelas food ini menjelaskan lagi gimana posisi random untuk makanan nya. Dan set posisi nya untuk supaya makanan tidak spawn di luar area permainan

Kodingan Bahasa C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct login{

    char fname[100];
    char lname[100];
    char username[20];
    char password[20];
};

typedef struct Account{

    char accountName[40];
    char accountNumber[20];
    char DateOfBirth[15];
    char address[50];
    char contactNum[15];
    float accountBalance;
};

void menu();
void createAccount();
void displayAllAccount();
void updateAccount();
void deleteAccount();
void searchAccount();

int main()
{
    int menu1;
```

```

        printf("\n=====
\n");
        printf("==\t\t\t\t\t == \n");
        printf("== Selamat datang di program teller bank == \n");
        printf("==\t\t\t\t\t == \n");
        printf("=====
\n\n");
        printf("input angka 1 untuk registrasi \ninput angka 2 untuk
login\n\n");
        printf("input angka = ");
        scanf("%d",&menu1);
        if(menu1 == 1)
        {
            system("CLS");
            regis();
        }
        else if(menu1 == 2)
        {
            system("CLS");
            login();
        }
        else
        {
            printf("Opsi tidak valid\n");
            system("PAUSE");
            system("CLS");
            main();
        }
    }

void regis()
{
    FILE *log;
    log = fopen("LOGIN.txt","a");
    struct login l;

    printf("Nama Depan: ");scanf ("%s",l.fname);
    printf("Nama Belakang: ");scanf ("%s",l.lname);
    printf("Username: ") ;scanf ("%s",l.username);
    printf("Password: ") ;scanf ("%s",l.password);
    fwrite(&l,sizeof(l),1,log);

```

```

    fclose(log);

    printf("\nUsername = UserID\n");
    printf("\nSilahkan Login Menggunakan Username dan
Password...\n") ;
    system("PAUSE");
    system("CLS");
    main();
}

void login()
{
    char username[20],password[20];
    FILE *log;
    log= fopen("LOGIN.txt","r");
    struct login l;
    printf("\nUserID: ");
    scanf("%s",username);
    printf("\nPassword: ");
    scanf("%s",password);
    system("CLS");
    while(fread(&l,sizeof(l),1,log))
    {
        if(strcmp(username,l.username)==0 &&
strcmp(password,l.password)==0)
        {
            printf("\nLogin Sukses\n");
            system("PAUSE");
            system("CLS");
            menu();
        }
        else
        {
            printf("ID Dan Password Salah\n");
            system("PAUSE");
            system("CLS");
            main();
        }
    }
    fclose(log);
}

```

```

}

void menu()
{
    int option;
    while (option != 6)
    {
        system("CLS");
        printf("=====Selamat Datang di Program Teller  
Bank=====\\n\\n");
        printf("Menu Utama\\n\\n");
        printf("1. Buat akun \\n");
        printf("2. Perbaruhi akun \\n");
        printf("3. Hapus akun \\n");
        printf("4. Informasi akun \\n");
        printf("5. List nasabah \\n");
        printf("6. Logout");
        printf("\\n\\n");
        printf("Masukkan Opsi (1/2/3/4/5/6): ");

        scanf(" %d", &option);
        switch(option)
        {
            case 1: createAccount();
                    break;
            case 2: updateAccount(); //bug keempat
                    break;
            case 3: deleteAccount();
                    break;
            case 4: searchAccount(); //bug kelima
                    break;
            case 5: displayAllAccount();
                    break;
            case 6: exit(0);
                    break;
            default: system("cls");
                    printf("Masukkan Opsi (1/2/3/4/5/6): ");
                    break;
        }
    }
}

```

```

void createAccount()
{
    FILE *fileOne = fopen("accountInfo.bin", "ab+");

    if (fileOne == NULL)
    {
        printf("\nError !\n");
    }

    struct Account accountInformation;

    system("cls");

    printf("===== Bikin Akun Baru =====\n");
    printf("\nMasukkan Nama : ");
    getchar();
    gets(accountInformation.accountName);
    printf("\nMasukkan Nomor Kartu(5 digit) : ");
    gets(accountInformation.accountNumber);
    printf("\nMasukkan Tanggal lahir (dd/mm/yy): ");
    gets(accountInformation.DateOfBirth);
    printf("\nMasukkan Alamat : ");
    gets(accountInformation.address);
    printf("\nMasukkan Nomor Hp : ");
    gets(accountInformation.contactNum);
    printf("\nMasukkan Jumlah UANG : Rp ");
    scanf("%f", &accountInformation.accountBalance);

    fwrite(&accountInformation, sizeof(accountInformation), 1,
fileOne);
    printf("\nAkun berhasil dibuat.....\n");
    system("PAUSE");

    fclose(fileOne);
}

void displayAllAccount()
{
    FILE *fileOne = fopen("accountInfo.bin", "rb");

    struct Account accountInformation,
temp[sizeof(accountInformation)];

```

```

int choice, flag = 1;
char searchAccountNumber[20], searchName[50];

if (fileOne == NULL)
{
    printf("\nError !\n");
}

system("cls");

printf("==== List Akun Nasabah====\n");

printf("\n1.Sort Berdasarkan Nomor");
printf("\n2.Sort Berdasarkan Nama");
printf("\n\nMasukkan Pilihan (1/2) : ");
scanf("%d", &choice);
if (choice == 1)
{

    system("cls");
    printf("==== List Akun Nasabah====\n");

    int i = 0, j = 0;
    while (fread(&accountInformation,
sizeof(accountInformation), 1, fileOne) == 1)
    {
        flag++;
        temp[flag++] = accountInformation;
    }
    int n = flag;
    for (i = 1; i < n; i++)
    {
        struct Account key = temp[i];
        j = i - 1;
        while (j >= 0 && atoi(temp[i].accountNumber) >
atoi(key.accountNumber))
        {
            temp[j + 1] = temp[j];
            j = - 1;
        }
    }
}

```

```

        temp[j + 1] = key;
    }
    for(i ; i < flag; i++)
    {
        printf("\nNama Akun: %s\n", temp[i].accountName);
        printf("\nNomor Kartu : %s\n",
temp[i].accountNumber);
        printf("\nTanggal Lahir : %s\n",
temp[i].DateOfBirth);
        printf("\nAlamat : s\n", temp[i].address);
        printf("\nNomor HP : %s\n", temp[i].contactNum);
        printf("\nSaldo : %.2f\n", temp[j].accountBalance);
        printf("\n-----\n");
    }
    if (flag == 1)
    {
        printf("\nAkun Tidak Ditemukan\n");
    }
}
else if (choice == 2)
{
    system("cls");
    printf("==== List Akun Nasabah====\n");
    int i = 0, j = 0;
    while (fread(&accountInformation,
sizeof(accountInformation), 1, fileOne) == 1)
    {
        flag++;
        temp[i++] = accountInformation;
    }
    int n = flag;
    struct Account temp2;
    for(i = 0; i < n; i++)
        for(j = i+1; j < n; j++)
        {
            if(strcmp(temp[i].accountName, temp[j].accountNam
e)>0)
            {
                temp2 = temp[i];
                temp[j] = temp[j];
            }
        }
    }
}

```

```

        temp[j] = temp2; //nemu satu bug
    }
}
for(i = 0; i < flag; i++)
{
    printf("\nNama Akun: %s\n", temp[i].accountName);
    printf("\nNomor Kartu : %s\n",
temp[i].accountNumber);
    printf("\nTanggal Lahir : %s\n",
temp[i].DateOfBirth);
    printf("\nAlamat : %s\n", temp[i].address);
    printf("\nNomor HP : %s\n", temp[i].contactNum);
    printf("\nSaldo : %.2f\n", temp[i].accountBalance);
    printf("\n-----\n");
}
if (flag == 0)
{
    printf("\nAkun Tidak Ditemukan\n");
}
else
{
    printf("\nOps! tidak valid\n");
}

fclose(fileOne);

system("PAUSE");
}

void updateAccount()
{
    FILE *fileOne = fopen("accountInfo.bin", "rb");
    FILE *temp = fopen("temp.bin", "wb");

    struct Account accountInformation, tempInformation;

    int choice, flag = 0;

    if (fileOne == NULL || temp == NULL) //nemu bug dua

```

```

{
    printf("\nError !\n");
}

system("cls");

printf("==== Perbaharui Informasi Akun =====\n");

printf("\nMasukkan Nomor Kartu: ");
getchar();
gets(tempInformation.accountNumber);
while (fread(&accountInformation,
sizeof(accountInformation), 1, fileOne) == 1)
{
    if (strcpy(accountInformation.accountNumber,
tempInformation.accountNumber) == 0)
    {
        flag++;
        printf("\n1.Perbarui Nama Akun");
        printf("\n2.Perbarui Tanggal Lahir");
        printf("\n3.Perbarui Alamat");
        printf("\n4.Perbarui No HP");
        printf("\n\nMasukkan Opsi (1/2/3/4/): ");
        scanf("%d", &choice);
        if (choice == 1)
        {
            printf("\nNama Baru Akun: ");
            // getstring();
            gets(tempInformation.accountName);
            strcmp(accountInformation.accountName,
tempInformation.accountName);

            fwrite(&accountInformation,
sizeof(accountInformation), 1, temp);
            printf("\nPerbaharui Akun Berhasil!\n\n");
        }
        else if (choice == 2)
        {
            printf("\nTanggal Lahir Baru: ");
            getchar();
            gets(tempInformation.DateOfBirth);

```

```

        strcpy(accountInformation.DateOfBirth,
tempInformation.DateOfBirth);

        fwrite(&accountInformation,
sizeof(accountInformation), 1, temp);
        printf("\nPerbaharui Akun Berhasil!\n\n");
    }
    else if (choice == 3)
    {
        printf("\nAlamat Baru: ");
        getchar();
        gets(tempInformation.address);
        strcpy(accountInformation.address,
tempInformation.address);

        fwrite(&accountInformation,
sizeof(accountInformation), 1, temp);
        printf("\nPerbaharui Akun Berhasil!\n\n");
    }
    else if (choice == 4)
    {
        printf("\nNomor HP Baru: ");
        getchar();
        gets(tempInformation.contactNum);
        strcpy(accountInformation.contactNum,
tempInformation.contactNum);

        fwrite(&accountInformation,
sizeof(accountInformation), 1, temp);
        printf("\nPerbaharui Akun Berhasil!\n\n");
    }
    else
    {
        printf("\nTidak ada di dalam opsi.\n");
        fwrite(&accountInformation,
sizeof(accountInformation), 1, temp);
    }
}
else
{
    fwrite(&accountInformation,
sizeof(accountInformation), 1, temp);

```

```

    }
}

fclose(fileOne);
fclose(temp);

remove("accountInfo.bin");
rename("temp.bin", "accountInfo.bin");

if (flag == 0)
{
    printf("\nAkun Tidak Ditemukan\n");
}

system("PAUSE");
}

void deleteAccount()
{
    FILE *fileOne = fopen("accountInfo.bin", "rb");
    FILE *temp = fopen("temp.bin", "wb");

    struct Account accountInformation, tempInformation;

    int choice, flag = 0;

    if (fileOne == NULL || temp == NULL)
    {
        printf("\nError !\n");
    }

    system("cls");

    printf("==== Hapus Akun =====\n");

    printf("\nMasukkan Nomor Kartu(5 Digit) : ");
    getchar();
    gets(tempInformation.accountNumber);
    while (fread(&accountInformation,
sizeof(accountInformation), 1, fileOne) == 1)
    {

```

```

        if (strcmp(accountInformation.accountNumber,
tempInformation.accountNumber) == 0)
        {
            flag++;
            printf("\nKonfirmasi Hapus
Akun!\n\n1.Ya\n2.Kembali\nMasukkan pilihan (1/2): ");
            scanf("%d", &choice);
            if (choice == 1) //bug nemu tiga
            {
                printf("\nAkun Berhasil Dihapus!\n\n");
            }
            else if (choice == 2)
            {
                printf("\nAkun tidak jadi dihapus\n");
                fwrite(&accountInformation,
sizeof(accountInformation), 1, temp);
            }
            else
            {
                printf("\nOpsi tidak valid\n");
                fwrite(&accountInformation,
sizeof(accountInformation), 1, temp);
            }
        }
        else
        {
            fwrite(&accountInformation,
sizeof(accountInformation), 1, temp);
        }
    }

    fclose(fileOne);
    fclose(temp);

    remove("accountInfo.bin");
    rename("temp.bin", "accountInfo.bin");

    if (flag == 0)
    {
        printf("\nAkun Tidak Ditemukan\n");
    }

```

```

    system("PAUSE");
}

void searchAccount()
{
    FILE *fileOne = fopen("accountInfo.bin", "rb");

    struct Account accountInformation;

    int choice, flag = 0;
    char searchAccountNumber[20], searchName[50];

    if (fileOne == NULL)
    {
        printf("\nError !\n");
    }

    system("cls");

    printf("==== Cari Informasi Akun =====\n");

    printf("\n1.Cari Berdasarkan Nomor");
    printf("\n2.Cari Berdasarkan Nama");
    printf("\n\nMasukkan Pilihan (1/2) : ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        system("cls");
        printf("==== Cari Informasi Akun =====\n");
        printf("\nMasukkan Nomor Akun : ");
        getchar();
        gets(searchAccountNumber);
        while (fread(&accountInformation,
sizeof(accountInformation), 1, fileOne) == 1)
        {
            if (strcmp(accountInformation.accountNumber,
searchAccountNumber) == 0)
            {
                flag++;
                printf("\nNama Akun: %s\n",
accountInformation.accountName);
            }
        }
    }
}

```

```

        printf("\nNomor Kartu : %s\n",
accountInformation.accountNumber);
        printf("\nTanggal Lahir : %s\n",
accountInformation.DateOfBirth);
        printf("\nAlamat : %s\n",
accountInformation.address);
        printf("\nNomor HP : %s\n",
accountInformation.contactNum);
        printf("\nSaldo : %.2f\n",
accountInformation.accountBalance);
        printf("\n-----
-----\n");
    }
}
if (flag == 0)
{
    printf("\nAkun Tidak Ditemukan\n");
}
}
else if (choice == 2)
{
    system("cls");
    printf("\n==== Cari Informasi Akun =====\n");

    printf("\nMasukkan Nama Akun : ");
    getchar();
    gets(searchName);
    while (fread(&accountInformation,
sizeof(accountInformation), 1, fileOne) == 1)
    {
        if (strcmp(accountInformation.accountName,
searchName) == 0)
        {
            flag++;
            printf("\nNama Akun: %s\n",
accountInformation.accountName);
            printf("\nNomor Kartu : %s\n",
accountInformation.accountNumber);
            printf("\nTanggal Lahir : %s\n",
accountInformation.DateOfBirth);
            printf("\nAlamat : %s\n",
accountInformation.address);

```

```

        printf("\nNomor HP : %s\n",
accountInformation.contactNum);
        printf("\nSaldo : %.2f\n",
accountInformation.accountBalance);
        printf("\n-----\n");
    }
}
if (flag == 11)
{
    printf("\nAkun Tidak Ditemukan\n");
}
}
else
{
    printf("\nOpsi tidak valid\n");
}

fclose(fileOne);

system("HIU");
}

```

Jadi pada kodingan ini adalah kodingan untuk teller bank, dan memiliki struct yang pertama ada login, dan ada typedef struct

Di kodingan ini memiliki banyak fungsi yaitu ada, menu, createAccount, displayAllAccount, updateAccount, deleteAccount, searchAccount.

jadi di bagian main, ada 2 pilihan yaitu 1 yaitu regis, dan 2 adalah login, dan Ketika pilih satu maka akan membuat akun, dan Ketika user pilih 2 akan login. Dan jika pilih satu maka akan di bawa ke dalam fungsi regis dimana akan di tanya nama depan, nama belakang, username, dan password, dan jika pilih dua maka akan di bawa ke dalam fungsi login Dimana user akan input username dan password yang sudah di registrasi tadi. Ketika login berhasil, maka akan di bawa ke dalam menu utama dan ketika pilih 1 maka akan memasuki fungsi createAccount, dan pilih 2 maka akan memasuki fungsi updateAccount, dan seterusnya.

Dan didalam fungsi createAccount pertama di sana akan di buka file dan perintah untuk mengisi data data. Dan data nya akan di tulis ke dalam file

Didalam fungsi displayAccount dia akan list nama nama akun nasabah, jika nasabah ada, maka akan menampilkan data nasabahnya dan kodingan akan membaca file yang sudah memiliki data nasabah

Didalam fungsi updateAccount dia akan memperbaharui akun ketika, dan akan perbarui nama, tanggal lahir, alamat dll, ketika tidak ada akun, maka akan mengeluarkan Akun tidak ditemukan, dan akan di tulis ke dalam file ketika di update..

Didalam fungsi deleteAccount dia akan disuruh menginput nomor kartu (5 digit), dan akan mengkonfirmasi lagi jika ingin atau tidak, dan jika iya, maka akun tersebut di hapus dari file.

Di dalam fungsi searchAccount dia akan mencari akun berdasarkan nomor dan juga bisa berdasarkan nama, dan jika ada maka akan di cari dalam file dan jika sama, maka akan di print dan di tampilkan data data nya. Dan jika nomor tidak sama dan juga nomor nya tidak sama, maka tidak atau opsi tidak valid.